```
!nvidia-smi
!nvcc --version
!pip install nvcc4jupyter
%load_ext nvcc4jupyter

%%cuda
#include <iostream>
#include <cuda_runtime.h>

using namespace std;

// CUDA Kernel for vector addition
__global__ void vectorAdd(const int* a, const int* b, int* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int n = 4;
    size_t size = n * sizeof(int);

    int *h_a = new int[n];
    int *h_b = new int[n];
    int *h_c = new int[n];

    int init_a[] = {1, 2, 3, 4};
    int init_b[] = {5, 6, 7, 8};

    for (int i = 0; i < n; i++) {
```

```
    h_a[i] = init_a[i];
    h_b[i] = init_b[i];
}

cout << "Vector A: ";
for (int i = 0; i < n; i++) cout << h_a[i] << " ";
cout << "\nVector B: ";
for (int i = 0; i < n; i++) cout << h_b[i] << " ";
cout << endl;

int *d_a, *d_b, *d_c;
cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc(&d_c, size);

cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

cout << "Result:" << endl;
for (int i = 0; i < n; i++) {
    cout << h_c[i] << " ";
}
cout << endl;

cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
```

```
    delete[] h_a; delete[] h_b; delete[] h_c;


    return 0;
}
```